CONCEPTUAL SERIES
TypeScript

Name: ______ Cohort/Batch: ______ Date: ______ Score: ______

TYPESCRIPT PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Programming Languages

TOTAL: 10 QUESTIONS

Q1 What are the primary design goals of TypeScript? Threat Model

Q2 How does TypeScript handle type checking: static or dynamic? Architecture

Q3 How does TypeScript manage memory? Configuration

Q4 What is TypeScript's concurrency model? Hardening

Q5 How are packages and dependencies managed in TypeScript? SSO/IdP

Q6 How does TypeScript implement object-oriented or functional programming? Observability

Q7 How does TypeScript handle errors? Lifecycle

Q8 What testing tools are commonly used in TypeScript? Compliance

Q9 What makes TypeScript well-suited for its target domain? Testing

Q10 What are common performance pitfalls in TypeScript? Tuning

ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 TypeScript was designed with specific priorities around Mitigation

TypeScript was designed with specific priorities around performance, safety, or developer productivity depending on its domain. These goals shape its syntax, type system, and standard library choices.

A2 TypeScript uses its type system to catch Primitives

TypeScript uses its type system to catch errors at compile time (static) or at runtime (dynamic). This affects refactoring safety, tooling support, and the kinds of bugs that reach production.

A3 TypeScript uses one of three approaches: manual Hardening

TypeScript uses one of three approaches: manual allocation/deallocation, a garbage collector that periodically reclaims unreachable objects, or automatic reference counting. Each has different latency and throughput tradeoffs.

A4 TypeScript supports concurrency through threads, Gotchas

TypeScript supports concurrency through threads, coroutines, async/await, or an actor model. The choice determines how I/O-bound and CPU-bound workloads are scaled and how shared state is safely accessed.

A5 TypeScript uses a package manager and a Federation

TypeScript uses a package manager and a manifest file to declare dependencies and version constraints. A lock file pins exact versions to ensure reproducible builds across environments.

A6 TypeScript supports classes and inheritance for OOP, Observability

TypeScript supports classes and inheritance for OOP, or first-class functions and immutability for FP, or both. Many modern TypeScript programs blend both paradigms depending on the problem.

A7 TypeScript surfaces errors via exceptions, Result/Either types, Automation

TypeScript surfaces errors via exceptions, Result/Either types, or error return values. The choice impacts whether errors are checked at compile time and how calling code is structured.

A8 TypeScript has a standard or widely adopted Governance

TypeScript has a standard or widely adopted test runner and assertion library. Tests are typically organized into unit, integration, and end-to-end layers with coverage reporting built in or via plugins.

A9 TypeScript excels in its domain due to Assessment

TypeScript excels in its domain due to a combination of performance characteristics, ecosystem maturity, language ergonomics, and community support that makes common tasks simple.

A10 Typical performance issues in TypeScript include excessive Optimization

Typical performance issues in TypeScript include excessive allocations, blocking the main thread or event loop, O(n^2) data structures, and missing caching. Profiling and benchmarking tools are available to diagnose them.